

Kafka-Based Saga Pattern (Spring Boot + Kafka)

ARCHITECTURE FLOW:

Order Service -> Payment Service -> Inventory Service

KAFKA TOPICS:

- order-created
- payment-success
- payment-failed
- inventory-reserved
- inventory-failed

COMMON EVENT MODEL

```
public record OrderEvent(Long orderId, String status) {}
```

ORDER PRODUCER

```
@Service
```

```
public class OrderProducer {  
    private final KafkaTemplate<String, OrderEvent> kafkaTemplate;  
  
    public OrderProducer(KafkaTemplate<String, OrderEvent> kafkaTemplate) {  
        this.kafkaTemplate = kafkaTemplate;  
    }  
  
    public void sendOrderCreated(Long orderId) {  
        kafkaTemplate.send("order-created",  
            new OrderEvent(orderId, "CREATED"));  
    }  
}
```

ORDER SERVICE

```
@Service
```

```
public class OrderService {  
    private final OrderProducer producer;  
    private final OrderRepository repository;  
  
    public OrderService(OrderProducer producer, OrderRepository repository) {  
        this.producer = producer;  
        this.repository = repository;  
    }  
  
    public void createOrder() {  
        Order order = new Order();  
        order.setStatus("CREATED");  
        repository.save(order);  
        producer.sendOrderCreated(order.getId());  
    }  
  
    public void cancelOrder(Long orderId) {  
        Order order = repository.findById(orderId).orElseThrow();  
        order.setStatus("CANCELLED");  
        repository.save(order);  
    }  
  
    public void completeOrder(Long orderId) {  
        Order order = repository.findById(orderId).orElseThrow();  
        order.setStatus("COMPLETED");  
        repository.save(order);  
    }  
}
```

ORDER CONSUMER

```
@Component
```

```
public class OrderConsumer {  
    private final OrderService orderService;  
  
    public OrderConsumer(OrderService orderService) {  
        this.orderService = orderService;  
    }  
  
    @KafkaListener(topics = "payment-failed")
```

```

    public void handlePaymentFailed(OrderEvent event) {
        orderService.cancelOrder(event.orderId());
    }

    @KafkaListener(topics = "inventory-reserved")
    public void handleInventoryReserved(OrderEvent event) {
        orderService.completeOrder(event.orderId());
    }
}

```

PAYMENT SERVICE

```

@Component
public class PaymentConsumer {
    private final KafkaTemplate<String, OrderEvent> kafkaTemplate;

    public PaymentConsumer(KafkaTemplate<String, OrderEvent> kafkaTemplate) {
        this.kafkaTemplate = kafkaTemplate;
    }

    @KafkaListener(topics = "order-created")
    public void processPayment(OrderEvent event) {
        boolean success = true;
        if (success) {
            kafkaTemplate.send("payment-success",
                new OrderEvent(event.orderId(), "PAID"));
        } else {
            kafkaTemplate.send("payment-failed",
                new OrderEvent(event.orderId(), "FAILED"));
        }
    }
}

```

INVENTORY SERVICE

```

@Component
public class InventoryConsumer {
    private final KafkaTemplate<String, OrderEvent> kafkaTemplate;

    public InventoryConsumer(KafkaTemplate<String, OrderEvent> kafkaTemplate) {
        this.kafkaTemplate = kafkaTemplate;
    }

    @KafkaListener(topics = "payment-success")
    public void reserveInventory(OrderEvent event) {
        kafkaTemplate.send("inventory-reserved",
            new OrderEvent(event.orderId(), "INVENTORY_RESERVED"));
    }
}

```

INTERVIEW SUMMARY:

- Choreography-based Saga
- Event-driven using Kafka
- Compensating transactions
- No distributed transactions